TDEW Estimation: PRAM Pseudocode and Complexity

Bucket-year anomaly local linear regression pipeline

Piero Palacios

2026-05-07

Table of contents

1 Purpose

This document describes the full algorithm used to estimate daily dewpoint temperature (`TD`) from minimum temperature (`TMIN`) for many spatial locations (`ID`). The design follows a PRAM-style view: independent operations are marked with `pardo`.

The project uses an anomaly local linear regression model:

$$TD_{anom}(t) = TD(t) - TD_{clim}(ID, doy)$$

$$TMIN_{anom}(t) = TMIN(t) - TMIN_{clim}(ID, doy)$$

For each spatial `ID` and each day-of-year `doy`, the model fits a weighted linear regression using `TMIN` anomaly and lagged anomaly variables.

2 PRAM Model

We use a CREW PRAM interpretation:

- Concurrent read: many processors may read shared input tables.
- Exclusive write: each processor writes to a disjoint output bucket, ID, or day-of-year result.
- Reductions, such as sums and means, are treated as parallel reductions.
- `pardo` means the loop body can be executed in parallel.

The bucket-year design is important because it prevents 300k independent ID tasks from repeatedly scanning the same historical parquet files.

3 Symbols

Symbol	Meaning
`N`	Number of spatial locations, approximately 300,000 IDs
`Y`	Number of training years, approximately 40
`D`	Days per year, at most 366
`T`	Total days per ID, `T = Y * D`
`R`	Total rows per climate variable, `R = N * T`
`B`	Number of ID buckets, for example 1024
`p`	Number of available processors or worker slots
`h`	Local DOY half-window, default `h = 11`
`W`	Samples per local regression, `W = O(Y * h)`
`F`	Number of regression features plus intercept, constant here
`Q`	Number of failed DOYs to repair, `Q <= D`
`H`	Forecast horizon in days

In this project, `D`, `h`, and `F` are small constants in practice, but they are shown explicitly in the complexity formulas.

4 Algorithm 1: Full TDEW Pipeline

```
Algorithm TDEW-PIPELINE
Input:
  TD monthly files for all IDs and dates
  TMIN monthly files for all IDs and dates
  Grid table containing the expected IDs
  Parameters: B buckets, h local DOY window, p processors

Output:
  Daily climatology per (ID, doy)
  Bucket-year merged training dataset
  Regression coefficients per (ID, doy)
```

```
  Optional repair coefficients for incomplete DOYs
  Optional TD predictions

1  climatology <- BUILD-CLIMATOLOGY(TD, TMIN)
2  bucketed_training <- BUILD-BUCKET-YEAR-DATASET(TD, TMIN, B)
3  bucketed_climatology <- SHARD-CLIMATOLOGY(climatology, B)
4  coefficients <- TRAIN-COEFFICIENTS(bucketed_training,
                                      bucketed_climatology,
                                      h,
                                      B)
5  incomplete_doys <- CHECK-COMPLETENESS(coefficients, Grid)
6  if incomplete_doys is not empty then
7      patch <- RERUN-FAILED-DOYS(bucketed_training,
                                  bucketed_climatology,
                                  incomplete_doys,
                                  h,
                                  B)
8      coefficients <- PATCH-COEFFICIENTS(coefficients,
                                          patch,
                                          incomplete_doys)
9      incomplete_doys <- CHECK-COMPLETENESS(coefficients, Grid)
10 end if
11 if forecasting is requested then
12     bucketed_future_tmin <- SHARD-FUTURE-TMIN(TMIN future, B)
13     predictions <- FORECAST-TD(coefficients,
                                  bucketed_climatology,
                                  bucketed_training,
                                  bucketed_future_tmin,
                                  H,
                                  B)
14 end if
15 return coefficients, predictions
```

5 Algorithm 2: Build Climatology

The climatology is the historical mean for each `(ID, doy)`.

```
Algorithm BUILD-CLIMATOLOGY
Input:
  TD rows:   (ID, date, TD)
  TMIN rows: (ID, date, TMIN)

Output:
  climatology rows: (ID, doy, TD_clim, TMIN_clim)

1  for each climate variable V in {TD, TMIN} pardo do
2      for each monthly file m of V pardo do
3          read rows from file m
4          for each row r pardo do
5              r.doy <- day_of_year(r.date)
```

```
6             emit partial record (r.ID, r.doy, r.value)
7         end for
8     end for
9     for each key (ID, doy) pardo do
10        sum_V(ID, doy) <- parallel sum of values with this key
11        count_V(ID, doy) <- parallel count of values with this key
12        clim_V(ID, doy) <- sum_V(ID, doy) / count_V(ID, doy)
13    end for
14 end for
15 join TD_clim and TMIN_clim by (ID, doy)
16 return climatology
```

Complexity:

$$W_{clim} = O(R)$$

$$T_p = O(R/p + \log R)$$

Space:

$$S_{clim} = O(ND)$$

6 Algorithm 3: Build Bucket-Year Training Dataset

This is the main redesign. The expensive monthly TD/TMIN merge is paid once. Rows are then distributed to deterministic ID buckets and compacted by year.

```
Algorithm BUILD-BUCKET-YEAR-DATASET
Input:
  TD monthly files
  TMIN monthly files
  Number of buckets B

Output:
  bucket-year training files:
  bucket b, year y -> rows (ID, date, TD, TMIN, doy)

1  for each year y pardo do
2      for each month m in year y pardo do
3          TD_m <- read TD rows for month m
4          TMIN_m <- read TMIN rows for month m
5          M_m <- hash join TD_m and TMIN_m by (ID, date)
6          for each row r in M_m pardo do
7              r.doy <- day_of_year(r.date)
8              r.bucket <- r.ID mod B
9              emit r to temporary partition (bucket = r.bucket, year = y)
10         end for
11     end for
12     for each bucket b in 0..B-1 pardo do
13         read all temporary rows for (bucket b, year y)
14         sort rows by (ID, date)
15         write compact file for (bucket b, year y)
```

```
16     end for
17 end for
18 return bucket-year training dataset
```

Complexity with hash joins:

$$W_{bucket} = O(R)$$

If sorting inside each bucket-year shard is included:

$$W_{bucket-sort} = O\left(R + \sum_{b,y} n_{b,y} \log n_{b,y}\right)$$

With balanced buckets, `n_{b,y} approx (N/B)D`, so:

$$W_{bucket-sort} = O\left(R + BY \cdot \frac{ND}{B} \log\left(\frac{ND}{B}\right)\right) = O\left(R \log\left(\frac{ND}{B}\right)\right)$$

Ideal parallel time:

$$T_p = O\left(\frac{R}{p} + \log\left(\frac{ND}{B}\right)\right)$$

Peak memory per bucket-year compaction:

$$S_{bucket-year} = O\left(\frac{ND}{B}\right)$$

7 Algorithm 4: Shard Climatology by Bucket

```
Algorithm SHARD-CLIMATOLOGY
Input:
  climatology rows (ID, doy, TD_clim, TMIN_clim)
  Number of buckets B

Output:
  bucketed climatology files

1  for each climatology row c pardo do
2      c.bucket <- c.ID mod B
3      emit c to bucket c.bucket
4  end for
5  for each bucket b in 0..B-1 pardo do
6      write climatology rows for bucket b
7  end for
8  return bucketed climatology
```

Complexity:

$$W_{clim-shard} = O(ND)$$

$$T_p = O(ND/p + \log B)$$

Peak memory with streaming:

$$S_{clim-shard} = O(ND/B)$$

If the full climatology is loaded at once, peak memory becomes `O(ND)`.

8 Algorithm 4b: Shard Future TMIN by Bucket

Forecasting needs the exogenous future `TMIN` over the prediction horizon laid out by the same buckets as the coefficients, so that each forecast task reads its inputs once. This mirrors the bucket-year preprocessing of Algorithm 3, but for a single variable and only over the forecast horizon.

Let `R_fut = N * H` be the number of future TMIN rows over the horizon `H`.

```
Algorithm SHARD-FUTURE-TMIN-BY-BUCKET
Input:
  future TMIN monthly files over the forecast horizon
  Number of buckets B

Output:
  bucketed future-TMIN files: bucket b -> rows (ID, date, TMIN)

1  for each month m in the horizon pardo do
2      read future TMIN rows for month m
3      for each row r pardo do
4          r.bucket <- r.ID mod B
5          emit r to temporary partition (bucket = r.bucket)
6      end for
7  end for
8  for each bucket b in 0..B-1 pardo do
9      write compact future-TMIN file for bucket b
10 end for
11 return bucketed future TMIN
```

Complexity:

$$W_{fut-shard} = O(R_{fut}) = O(NH)$$

$$T_p = O(NH/p + \log B)$$

Peak memory with streaming:

$$S_{fut-shard} = O(NH/B)$$

9 Algorithm 5: Train Anomaly Coefficients

For each `ID` and each target `doy`, fit a weighted regression over a circular day-of-year neighborhood.

```
Algorithm TRAIN-COEFFICIENTS
Input:
  bucket-year training dataset
  bucketed climatology
  Local DOY half-window h
  Number of buckets B

Output:
```

```
    coefficients per (ID, doy)

 1  for each bucket b in 0..B-1 pardo do
 2      train_b <- read all bucket-year training rows for bucket b
 3      clim_b <- read climatology rows for bucket b
 4      for each ID i in bucket b pardo do
 5          X_i <- rows of train_b for ID i
 6          C_i <- rows of clim_b for ID i
 7          sort X_i by date
 8          for each row t in X_i pardo do
 9              TD_anom(t) <- TD(t) - TD_clim(i, doy(t))
10              TMIN_anom(t) <- TMIN(t) - TMIN_clim(i, doy(t))
11          end for
12          for each row t in X_i pardo do
13              create lag features:
14                TD_anom_lag1(t), TD_anom_lag2(t), TMIN_anom_lag1(t)
15          end for
16          for each target day d in 1..366 pardo do
17              N_d <- rows where circular_distance(doy(t), d) <= h
18              remove rows in N_d with missing target or missing features
19              if size(N_d) >= minimum_sample_count then
20                  for each row t in N_d pardo do
21                      weight(t) <- tricube(circular_distance(doy(t), d) / h)
22                  end for
23                  beta <- weighted_least_squares(N_d)
24                  write coefficients for (ID i, doy d)
25              end if
26          end for
27      end for
28      write coefficient file for bucket b
29      write failure log for bucket b, if needed
30  end for
31  return coefficient dataset
```

Per local regression:

$$W_{one-fit} = O(WF^2 + F^3)$$

Here `F` is constant, so:

$$W_{one-fit} = O(W) = O(Yh)$$

All coefficient training:

$$W_{train} = O(NDW) = O(NDYh)$$

Ideal PRAM span if buckets, IDs, and DOYs are fully parallel:

$$T_\infty = O(\log W + F^3)$$

With `p` processors:

$$T_p = O(NDYh/p + \log(Yh))$$

Practical bucket-parallel execution has at most `B` outer tasks, so:

$$T_p = O\left(\frac{NDYh}{\min(p, B)} + \text{I/O overhead}\right)$$

Peak memory per bucket task:

$$S_{train-bucket} = O\left(\frac{NT}{B} + \frac{ND}{B}\right)$$

The first term is the bucket training time series. The second term is the bucket climatology.

10 Algorithm 6: Check Completeness

```
Algorithm CHECK-COMPLETENESS
Input:
  coefficients
  expected ID count from grid

Output:
  incomplete DOYs

1  expected_count <- number of IDs in grid
2  for each coefficient row c pardo do
3      emit key c.doy with value c.ID
4  end for
5  for each doy d in 1..366 pardo do
6      observed_count(d) <- distinct count of IDs emitted for d
7      if observed_count(d) < expected_count then
8          mark d as incomplete
9      end if
10 end for
11 return incomplete DOYs
```

Let `C = ND` be the complete number of coefficient rows.

$$W_{check} = O(C)$$

$$T_p = O(C/p + \log C)$$

Space:

$$S_{check} = O(C)$$

The output itself is small: at most `D` incomplete DOYs.

11 Algorithm 7: Repair Failed DOYs

Only the incomplete DOYs are retrained. If `Q` is small, this is much cheaper than retraining all 366 DOYs.

```
Algorithm RERUN-FAILED-DOYS
Input:
  bucket-year training dataset
  bucketed climatology
  incomplete DOYs Q
```

```
    Local DOY half-window h

  Output:
    patch coefficient dataset

  for each bucket b in 0..B-1 pardo do
      train_b <- read all bucket-year training rows for bucket b
      clim_b <- read climatology rows for bucket b
      for each ID i in bucket b pardo do
          recompute anomalies and lags for ID i
          for each failed day d in Q pardo do
              fit weighted local regression for (ID i, doy d)
              write patch coefficients for (ID i, doy d)
          end for
      end for
  end for
  return patch coefficient dataset
```

Complexity:

$$W_{repair} = O(NQYh)$$

$$T_p = O(NQYh/p + \log(Yh))$$

If `Q << D`, repair is much faster than full training.

12 Algorithm 8: Patch Coefficients

```
  Algorithm PATCH-COEFFICIENTS
  Input:
    base coefficient dataset
    patch coefficient dataset
    incomplete DOYs Q

  Output:
    corrected coefficient dataset

  for each bucket b in 0..B-1 pardo do
      base_b <- read coefficient rows for bucket b
      patch_b <- read patch rows for bucket b
      base_keep <- rows in base_b whose doy is not in Q
      corrected_b <- union(base_keep, patch_b)
      remove duplicate (ID, doy), keeping the patch row
      sort corrected_b by (ID, doy)
      write corrected coefficients for bucket b
  end for
  return corrected coefficient dataset
```

Complexity:

$$W_{patch} = O(ND + NQ)$$

$$T_p = O((ND + NQ)/p)$$

Peak memory per bucket:

$$S_{patch-bucket} = O(ND/B + NQ/B)$$

13 Algorithm 9: Optional Forecasting (bucketed)

Forecasting is parallel across buckets, and across IDs within a bucket, but recursive through time for each ID: day `t` depends on the previously predicted TD anomaly, so the horizon loop cannot be parallelized within one ID. Like training, the work is organized by bucket so that each coefficient, climatology, history, and future-TMIN shard is read exactly once. The future-TMIN shards are produced by Algorithm 4b; the history (used only to seed the first two lags) is read from the bucket-year training dataset.

```
Algorithm FORECAST-TD
Input:
  bucketed coefficients per (ID, doy)
  bucketed climatology per (ID, doy)
  bucketed history (to seed lags) and bucketed future TMIN
  forecast horizon H
  Number of buckets B

Output:
  predicted TD for each (ID, date)

1  for each bucket b in 0..B-1 pardo do
2      coeff_b <- read coefficient rows for bucket b
3      clim_b  <- read climatology rows for bucket b
4      hist_b  <- read history rows for bucket b
5      ftmin_b <- read future TMIN rows for bucket b
6      for each ID i in bucket b pardo do
7          initialize lagged TD/TMIN anomalies from hist_b(i)
8          for forecast day t from 1 to H do          // sequential in t
9              d <- day_of_year(date_t)
10             beta <- coeff_b(i, d)
11             x <- current TMIN anomaly and lagged anomaly features
12             TD_anom_hat <- beta dot x
13             TD_hat <- clim_b(i, d) + TD_anom_hat
14             update lagged anomalies with TD_anom_hat and TMIN_anom(t)
15             write prediction (ID i, date_t, TD_hat)
16         end for
17     end for
18     write prediction file for bucket b
19 end for
20 return predictions
```

Complexity:

$$W_{forecast} = O(NH)$$

Practical bucket-parallel time (at most `B` outer tasks, recursive over the horizon within each ID):

$$T_p = O\left(\frac{NH}{\min(p, B)} + H\right)$$

The `+ H` term remains because the forecast for one ID is recursive over time. As with training, the bucket structure bounds the effective parallelism by `p_{eff} \leq B`.

Peak memory per bucket task:

$$S_{forecast-bucket} = O\left(\frac{NH}{B} + \frac{ND}{B}\right)$$

14 Old Design vs Bucket-Year Design

The original Colab-style design trained one task per `ID`. Each task discovered, opened, and filtered the same monthly parquet files.

Let `S = 12Y` be the number of monthly source files per variable.

Old I/O task overhead:

$$O(NS)$$

For `N = 300,000` and `Y = 40`, this means roughly:

$$300,000 \times 480 = 144,000,000$$

file-touch attempts per variable family before considering actual row reads.

The bucket-year design changes the I/O pattern:

- Source files are scanned once during preprocessing.
- Training reads compact bucket-year files.
- Workers write disjoint bucket outputs.
- No large driver-side concatenation is required.

New source scan overhead:

$$O(S)$$

New model training work:

$$O(NDYh)$$

So the redesign does not remove the statistical work of fitting many local regressions, but it removes the repeated source-file scanning that made the previous implementation slow and fragile.

15 Summary Complexity Table

Phase	Sequential work	Ideal parallel time with `p` processors	Peak memory target
Build climatology	`O(R)`	`O(R/p + log R)`	`O(ND)` output
Build bucket-year dataset	`O(R)` or `O(R log(ND/B))` with sorting	`O(R/p + log(ND/B))`	`O(ND/B)` per compaction
Shard climatology	`O(ND)`	`O(ND/p + log B)`	`O(ND/B)` streaming
Shard future TMIN	`O(NH)`	`O(NH/p + log B)`	`O(NH/B)` streaming
Train coefficients	`O(NDYh)`	`O(NDYh/min(p,B) + log(Yh))`	`O(NT/B + ND/B)` per bucket

Phase	Sequential work	Ideal parallel time with `p` processors	Peak memory target
Check completeness	`O(ND)`	`O(ND/p + log(ND))`	`O(ND)` or bucket streaming
Repair `Q` failed DOYs	`O(NQYh)`	`O(NQYh/min(p,B) + log(Yh))`	same as training bucket
Patch coefficients	`O(ND + NQ)`	`O((ND + NQ)/p)`	`O(ND/B + NQ/B)` per bucket
Forecast horizon `H`	`O(NH)`	`O(NH/min(p,B) + H)`	`O(NH/B)` per output bucket

16 Overall Complexity

The full pipeline is a sequence of parallel phases, so the total work is the sum of the work of each phase. The dominant terms are:

- bucket-year preprocessing,
- full coefficient training,
- optional failed-DOY repair,
- optional forecasting.

Using `R = NYD`, the total sequential work for preprocessing plus full training without repair and without forecasting is:

$$W_{total} = O\left(R \log\left(\frac{ND}{B}\right) + NDYh\right)$$

Equivalently:

$$W_{total} = O\left(NYD \log\left(\frac{ND}{B}\right) + NDYh\right)$$

If sorting cost inside bucket-year files is ignored or treated as an implementation detail, the simpler expression is:

$$W_{total} = O(NYD + NDYh) = O(NDYh)$$

because `h` is the main local-regression multiplier.

If repair is needed for `Q` failed DOYs, add:

$$W_{repair-total} = O(NQYh)$$

So full training plus repair is:

$$W_{total+repair} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D + Q)\right)$$

If forecasting is also executed for horizon `H`, add:

$$W_{forecast-total} = O(NH)$$

Therefore the most complete end-to-end expression is:

$$W_{end-to-end} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D + Q) + NH\right)$$

For parallel execution, the phases are sequentially dependent, but each phase uses `pardo` internally. With `p` processors and `B` bucket tasks, the practical parallel time is:

$$T_p = O\left(\frac{NYD\log(ND/B)}{p} + \frac{NDYh}{\min(p,B)} + \frac{NQYh}{\min(p,B)} + \frac{NH}{\min(p,B)} + H + \log(NYD)\right)$$

The `+ H` term remains because forecasting is recursive over time for each ID. If forecasting is not executed, remove `NH/min(p,B) + H`. If no failed DOYs exist, remove the `NQYh / min(p,B)` term.

For this project, the practical bottleneck after the bucket-year redesign is:

$$O(NDYh)$$

That is the cost of fitting local regressions for many IDs and all days of year. The redesign mainly reduces repeated I/O from the old Colab-style approach.

17 HPC Interpretation

The total work complexity does not depend on `p` because it measures the total number of operations required by the algorithm. The same joins, anomaly calculations, local regressions, repairs, and forecasts must be computed whether the job runs on one processor or on a university high performance computing cluster.

On an HPC system, the relevant measure for execution time is the parallel runtime:

$$T_p = O\left(\frac{W}{p_{eff}} + \text{I/O overhead} + \text{scheduling overhead} + \text{sequential dependencies}\right)$$

where `p_eff` is the effective number of processors that the algorithm can use. It can be smaller than the physical number of available cores because of bucket count, memory bandwidth, file-system bandwidth, scheduler overhead, and dependencies between pipeline stages.

For the current bucket-based training design:

$$p_{eff} \leq B$$

This is because the main training stage creates one large parallel task per bucket. If `B = 1024`, the training stage can naturally expose up to about 1024 parallel bucket tasks. If the HPC provides fewer than 1024 worker slots, the cluster can be used efficiently. If the HPC provides far more than 1024 worker slots, the extra processors will not reduce training time much unless we increase the number of buckets or add another level of parallelism inside each bucket, for example by splitting work by `ID` or by groups of `doy`.

A practical rule for an HPC deployment is:

$$B \geq 4p$$

where `p` is the number of worker slots planned for the run. This gives the scheduler enough bucket tasks to balance slow and fast workers. However, `B` should not be made arbitrarily large because too many small buckets create many small files and increase scheduling overhead.

Example starting points:

Planned worker slots	Suggested bucket count
64	512 or 1024
128	1024 or 2048
256	2048 or 4096
512	4096 or 8192

Therefore, for a university HPC, the project should be evaluated using both:

- total work, which describes the size of the scientific computation; and
- parallel runtime, which describes how much wall-clock time can be reduced by using the cluster.

The expected speedup is strong while `p_eff` increases, but it eventually saturates because of I/O bandwidth, scheduler overhead, bucket count, and the recursive forecast dependency over the horizon `H`.

18 Practical Notes for This Machine

With about 300k IDs, 40 years, and 32 logical CPU threads, the best strategy is to keep the bucket count much larger than the number of workers. For example, `B = 1024` gives roughly `N/B approx 293` IDs per bucket, which keeps each bucket task large enough to amortize overhead but small enough to fit in memory.

18.1 Hardware is just more processors

The complexity above is hardware-agnostic: the total work `W = O(NDYh)` is the same whether the pipeline runs on CPU cores or on a GPU. Hardware changes only two things: the number of parallel processors `p` it exposes, and the constant factor `c` of a single local fit. The parallel runtime is always

$$T_p = c \cdot \frac{W}{p_{eff}} + \text{overhead} \ .$$

The per-`(ID, doy)` weighted least squares is embarrassingly parallel: there are about `N * D` (roughly `1.1 \times 10^8`) independent fits, each a small `F = 5` feature weighted regression over `W = O(Yh)` neighborhood rows. This maps equally well to:

- **CPU**: many single-threaded bucket worker processes (one per core), where the bucket structure bounds `p_{eff} \leq B`; or
- **GPU**: a device that evaluates thousands of these tiny fits concurrently once they are batched, by assembling the `F \times F` weighted normal equations `(X^\top W X)\beta = X^\top W y` for many `(ID, doy)` pairs and solving them in a single batched call.

So a GPU is, in PRAM terms, simply a source of a larger `p` (with a different constant `c`). Which of CPU or GPU yields the smaller `c \cdot T_p` for this workload is an empirical question, answered by the scaling experiments below, not by the complexity model.

The remaining, hardware-independent, speedups still come from:

- avoiding repeated parquet scans (the bucket-year redesign),
- using bucket-year files,
- keeping worker memory bounded,
- parallelizing by bucket,
- and repairing only failed DOYs instead of repeating all training.

19 Experimental Methodology

This section defines how the implementation is evaluated on the university HPC. The goal is to measure, as a function of the number of processors `p`, three standard parallel-performance quantities, and to do so on both CPU and GPU hardware and on both dataset versions.

19.1 Quantities measured

For a fixed problem size, let `T(p)` be the wall-clock time of a phase (or of the end-to-end pipeline) using `p` processors. We report:

$$\text{Execution time: } T(p) \qquad \text{Speedup: } S(p) = \frac{T(1)}{T(p)} \qquad \text{Efficiency: } E(p) = \frac{S(p)}{p}$$

S(p) is ideally linear (S(p) = p) and E(p) is ideally 1. The PRAM analysis predicts the departure from ideal: because the training and forecast stages are bucket-parallel with p_{eff} \leq B, efficiency stays high while p \leq B and then degrades; I/O bandwidth, scheduler overhead, and the sequential pipeline dependencies set the practical ceiling.

19.2 Definition of a "processor" p

To compare CPU and GPU on one axis, p counts independent worker slots:

Hardware	One processor (p unit)	How p is varied
CPU (SLURM)	one single-threaded Dask worker process (one core, BLAS pinned to 1 thread)	dask_jobqueue.SLURMCluster scaled to p workers
GPU (A100)	one A100 GPU (one dask-cuda worker)	p = number of A100s in the allocation

Within a single GPU, the batched weighted-least-squares solver uses all CUDA cores; that intra-device parallelism is held fixed and absorbed into the constant c, so the reported p axis is the number of GPUs. This keeps S(p) and E(p) meaningful across hardware: both answer "how much faster with p independent workers".

19.3 Controls

To make the measurements reproducible and fair:

- **Baseline T(1)** is a single worker with threads_per_worker = 1 and BLAS thread pools pinned to 1 (OMP_NUM_THREADS=1, etc.), as in run_pipeline.sh.
- **Bucket count** is held at B \geq 4\,p_{max} for the whole sweep so the bucket granularity is never the limiting factor before p reaches its maximum.
- **Repeated trials**: each (p, n) point is run k times (e.g. k = 3) and the median wall-clock is reported, to suppress filesystem and scheduler jitter.
- **Storage**: worker scratch / spill goes to node-local fast storage (local_directory = $TMPDIR); input parquet lives on the shared filesystem.
- **Warm vs cold cache** is recorded; cold-cache (first-touch) runs are reported separately because the I/O-bound preprocessing phases are cache sensitive.
- Only the phase under test is timed (with time.perf_counter around the driver call); inputs from earlier phases are precomputed once and reused.

19.4 Strong scaling (fixed problem, more processors)

Hold the problem size fixed at the full grid (or a fixed representative subset of N IDs) and vary p:

$$p \in \{1, 2, 4, 8, 16, 32, \ldots\} \quad \text{(CPU)} \qquad p \in \{1, 2, 4, 8\} \quad \text{(A100 GPUs)}$$

For each p we record T(p) for the dominant **train-coefficients** phase (and, separately, the I/O-bound build-bucket and climatology phases, and optionally forecast). From these we compute S(p) and E(p) and plot all three against p. The expected curve rises near-linearly while p \leq B and the workload is compute-bound, then flattens — directly testing the p_{eff} \leq B prediction.

19.5 Weak scaling (more processors, proportionally larger problem)

Hold the work per processor constant by scaling the number of trained IDs with p:

$$n(p) = n_0 \cdot p$$

where n is the number of IDs trained. The ID subset is selected deterministically through the existing bucket_ids parameter (each bucket holds \approx N/B IDs, so choosing \lceil 4p \rceil buckets gives a controlled n(p)). Weak-scaling efficiency is

$$E_{weak}(p) = \frac{T(1, n_0)}{T(p, n_0 p)}$$

which is ideally `1`. Because the per-`(ID, doy)` fits are independent, the compute part of training is expected to weak-scale well; departures isolate I/O and scheduling overhead that grow with the number of workers.

19.6 Datasets: PISCOt v1.0 vs v2.0

The full strong- and weak-scaling protocol is run on both dataset versions. The two versions are supplied to the same pipeline through the input path / variable-folder parameters (`--base`, `--td-var`, `--tmin-var`) writing to separate result roots, so no code differs between runs. We compare:

- **Performance**: `T(p)`, `S(p)`, `E(p)` curves for v1.0 vs v2.0 (the work `W` differs only if `N`, `Y`, or data density differ between versions).
- **Science**: distributions of the fitted coefficients and of the prediction skill, to assess how the dataset version affects the estimated model, independent of runtime.

19.7 Hardware configurations

Configuration	Cluster	`p` swept
CPU multi-node	`dask_jobqueue.SLURMCluster` (one process per core, `B \geq 4p_{max}`)	worker processes
GPU single/multi-A100	`dask-cuda` (`LocalCUDACluster` or SLURM `--gres=gpu:a100:p`)	number of A100s

19.8 Reporting

For each (dataset × hardware) combination we produce: (1) a table of `T(p)`, `S(p)`, `E(p)` for the strong-scaling sweep; (2) a table of `E_{weak}(p)`; and (3) three plots — execution time vs `p`, speedup vs `p` (with the ideal `S(p)=p` reference), and efficiency vs `p`. These quantify both how the implementation scales and which hardware minimizes wall-clock time for this workload.